Assignment – 4

Tanu B
261100690012

Hard link vs. soft link

Part A — Create the links
Create a directory named link_lab and navigate to it.
Create a file named security.txt containing the following text:
Linux File System Security Lab
Confidential Security Report

Create:
a hard link named security_hard.txt
a symbolic link named security_soft.txt

List all three files using a command that displays their inode numbers.

Record the inode numbers of:
security.txt : 1718373
security_hard.txt : 1718373
security_soft.txt : 1718375

What difference do you observe?

```
hdoop@Ubuntu:~/Desktop$ cd Assignment4
hdoop@Ubuntu:~/Desktop/Assignment4$ ls
hdoop@Ubuntu:~/Desktop/Assignment4$ mkdir links
hdoop@Ubuntu:~/Desktop/Assignment4$ cd links
hdoop@Ubuntu:~/Desktop/Assignment4/links$ gedit security.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$ ln security.txt security_hard.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$ ln -s security.txt security_soft.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$ ls -li
total 8
1718373 -rw-rw-r-- 2 hdoop hdoop 502 Sep 13 12:37 security_hard.txt
1718375 lrwxrwxrwx 1 hdoop hdoop  12 Sep 13 12:38 security_soft.txt -> security.txt
1718373 -rw-rw-r-- 2 hdoop hdoop 502 Sep 13 12:37 security.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$
```

Difference Observed: The original file (security.txt) and the hard link (security_hard.txt) share the exact same inode number. The symbolic link (security_soft.txt) has a different, unique inode number.

Part B — Examine File Metadata

Use ls -l to examine the three files. Identify the file type of each file from the first character of the permission field.
Compare the link count of the original file, hard link, and symbolic link. Explain the observed values.
Use the stat command on all three files and record the following information:
Parameter Original Hard Link Soft Link

Inode number
File type
Size
Link count
Permissions
Compare the file sizes of the original file, hard link, and symbolic link. Why might the size of the symbolic link differ from the original file?

1. File Types (from ls -l):

- **security.txt:** Regular file (indicated by -)
- **security_hard.txt:** Regular file (indicated by -)
- **security_soft.txt:** Symbolic link (indicated by l)

2. Link Count Comparison:

- The original file and hard link both show a link count of **2**. This is because both filenames point to the exact same underlying inode (data blocks). Creating a hard link increments the reference count on the inode.
- The symbolic link shows a link count of **1**. A symbolic link is a distinct file pointing to a path, so it has its own inode and does not increment the link count of the target file.

```
hdoop@Ubuntu:~/Desktop/Assignment4/links$ ls -l
total 8
-rw-rw-r-- 2 hdoop hdoop 502 Sep 13 12:37 security_hard.txt
lrwxrwxrwx 1 hdoop hdoop  12 Sep 13 12:38 security_soft.txt -> security.txt
-rw-rw-r-- 2 hdoop hdoop 502 Sep 13 12:37 security.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$ stat security.txt
  File: security.txt
  Size: 502             Blocks: 8          IO Block: 4096   regular file
Device: 803h/2051d      Inode: 1718373     Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1001/   hdoop)   Gid: ( 1001/   hdoop)
Access: 2026-09-13 12:37:30.531110280 +0530
Modify: 2026-09-13 12:37:30.551120279 +0530
Change: 2026-09-13 12:37:57.765719090 +0530
 Birth: 2026-09-13 12:37:30.531110280 +0530
hdoop@Ubuntu:~/Desktop/Assignment4/links$ stat security_hard.txt
  File: security_hard.txt
  Size: 502             Blocks: 8          IO Block: 4096   regular file
Device: 803h/2051d      Inode: 1718373     Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1001/   hdoop)   Gid: ( 1001/   hdoop)
Access: 2026-09-13 12:37:30.531110280 +0530
Modify: 2026-09-13 12:37:30.551120279 +0530
Change: 2026-09-13 12:37:57.765719090 +0530
 Birth: 2026-09-13 12:37:30.531110280 +0530
hdoop@Ubuntu:~/Desktop/Assignment4/links$ stat security_soft.txt
  File: security_soft.txt -> security.txt
  Size: 12              Blocks: 0          IO Block: 4096   symbolic link
Device: 803h/2051d      Inode: 1718375     Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1001/   hdoop)   Gid: ( 1001/   hdoop)
Access: 2026-09-13 12:38:33.288469388 +0530
Modify: 2026-09-13 12:38:14.781221536 +0530
Change: 2026-09-13 12:38:14.781221536 +0530
 Birth: 2026-09-13 12:38:14.781221536 +0530
```

Parameter	Original (security.txt)	Hard Link (security_hard.txt)	Soft Link (security_soft.txt)
Inode number	Identical to Hard Link	Identical to Original	Unique
File type	regular file	regular file	symbolic link
Size	502 bytes	502 bytes	12 bytes
Link count	2	2	1
Permissions	-rw-rw-r--	-rw-rw-r--	lrwxrwxrwx

4. File Size Explanation: The original file and the hard link have identical sizes (502 bytes) because they point to the exact same text data. The symbolic link has a size of 12 bytes because a symbolic link simply stores the text path of its target ("security.txt", which is exactly 12 characters/bytes long), not the target's data.

Part C — Investigate readlink
Execute readlink on security_soft.txt. What information is displayed?
Execute readlink on security_hard.txt. What do you observe?
Based on your observations, explain why readlink works differently for hard links and symbolic links.
Use:
readlink -f security_soft.txt

What does the -f option reveal?

```
hdoop@Ubuntu:~/Desktop/Assignment4/links$ readlink security_soft.txt
security.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$ readlink security_hard.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$ readlink -f security_hard.txt
/home/hdoop/Desktop/Assignment4/links/security_hard.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$ readlink -f security_soft.txt
/home/hdoop/Desktop/Assignment4/links/security.txt
```

Explanation: The readlink command is specifically designed to read the value of a symbolic link (the path it points to). It works on the soft link because the soft link contains a path string. It fails on the hard link because a hard link is not a pointer to a path; it is simply a direct filesystem reference to an inode, just like the original file.

The -f option: readlink -f security_soft.txt reveals the full absolute canonical path (e.g., /home/hdoop/Desktop/Assignment4/links/security.txt) rather than just the relative path stored in the link.

Part D — Content Modification

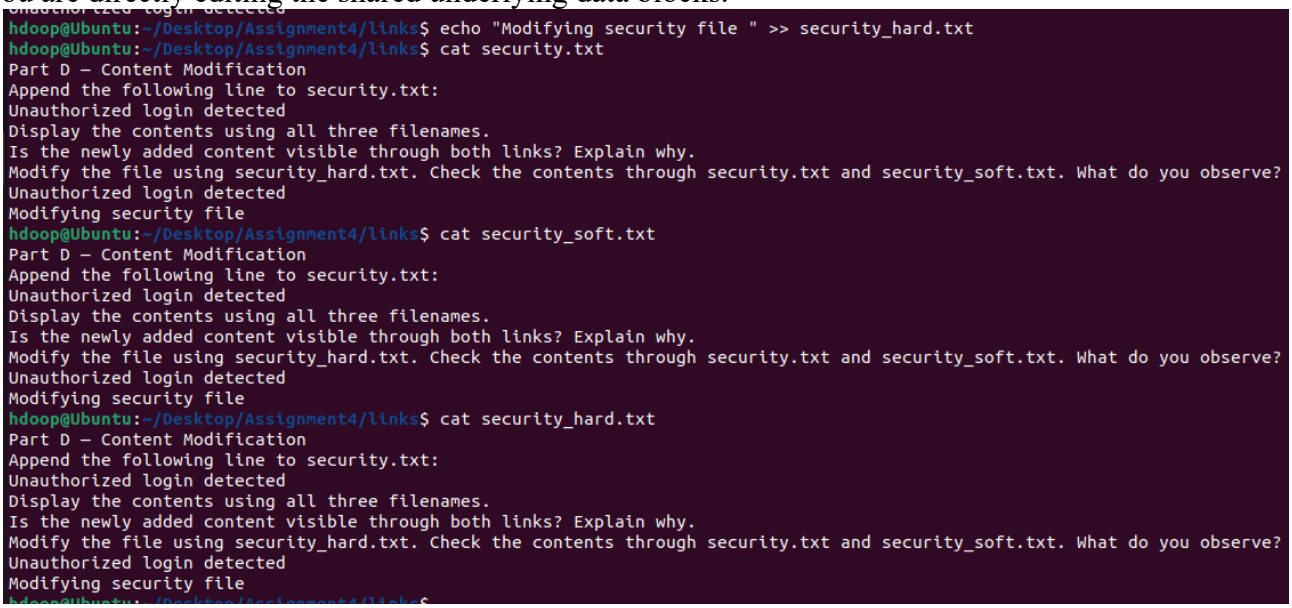
Append the following line to security.txt:
Unauthorized login detected
Display the contents using all three filenames.
Is the newly added content visible through both links? Explain why.
Modify the file using security_hard.txt. Check the contents through security.txt and security_soft.txt. What do you observe?

```
hdoop@Ubuntu:~/Desktop/Assignment4/links$ echo "Unauthorized login detected" >> security.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$ cat security.txt
Part D – Content Modification
Append the following line to security.txt:
Unauthorized login detected
Display the contents using all three filenames.
Is the newly added content visible through both links? Explain why.
Modify the file using security_hard.txt. Check the contents through security.txt and security_soft.txt. What do you observe?
Unauthorized login detected
hdoop@Ubuntu:~/Desktop/Assignment4/links$ cat security_hard.txt
Part D – Content Modification
Append the following line to security.txt:
Unauthorized login detected
Display the contents using all three filenames.
Is the newly added content visible through both links? Explain why.
Modify the file using security_hard.txt. Check the contents through security.txt and security_soft.txt. What do you observe?
Unauthorized login detected
hdoop@Ubuntu:~/Desktop/Assignment4/links$ cat security_soft.txt
Part D – Content Modification
Append the following line to security.txt:
Unauthorized login detected
Display the contents using all three filenames.
Is the newly added content visible through both links? Explain why.
Modify the file using security_hard.txt. Check the contents through security.txt and security_soft.txt. What do you observe?
Unauthorized login detected
```

The newly added content is visible through **all three** filenames.

- **Explanation:** Modifying the original file alters the data at its inode. The hard link points directly to that same inode, and the soft link points to the filename that resolves to that inode. Therefore, the changes are universally reflected.

Modifying the file via **security_hard.txt** similarly updates the content across all three files because you are directly editing the shared underlying data blocks.



Part E — Rename Experiment

Rename:
security.txt

to:

incident.txt
Try accessing the content through security_hard.txt. Does it still work?
Try accessing the content through security_soft.txt. Does it still work?
Use readlink security_soft.txt after renaming the original file. What path is still stored in the symbolic link?
Explain why renaming the original file affects the symbolic link differently from the hard link.

: Accessing via security_hard.txt: Still works perfectly. Hard links are bound directly to the inode (the data itself), completely independent of the original filename.

- **Accessing via security_soft.txt:** Fails with a "No such file or directory" error.
- **readlink security_soft.txt Output:** Still outputs security.txt.
- **Explanation:** Renaming the original file breaks the symbolic link because the soft link is strictly a text pointer to the string name "security.txt". Once that name disappears, the soft link is orphaned. The hard link remains unaffected because it maps to the underlying inode number, which does not change during a rename operation.

```
hdoop@Ubuntu:~/Desktop/Assignment4/links$ mv security.txt incident.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$ cat security.txt
cat: security.txt: No such file or directory
hdoop@Ubuntu:~/Desktop/Assignment4/links$ cat incident.txt
Part D – Content Modification
Append the following line to security.txt:
Unauthorized login detected
Display the contents using all three filenames.
Is the newly added content visible through both links? Explain why.
Modify the file using security_hard.txt. Check the contents through security.txt and security_soft.txt. What do you observe?
Unauthorized login detected
Modifying security file
hdoop@Ubuntu:~/Desktop/Assignment4/links$ cat security_hard.txt
Part D – Content Modification
Append the following line to security.txt:
Unauthorized login detected
Display the contents using all three filenames.
Is the newly added content visible through both links? Explain why.
Modify the file using security_hard.txt. Check the contents through security.txt and security_soft.txt. What do you observe?
Unauthorized login detected
Modifying security file
hdoop@Ubuntu:~/Desktop/Assignment4/links$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
hdoop@Ubuntu:~/Desktop/Assignment4/links$ readlink security_soft.txt
security.txt
```

Part F — Deletion Experiment

Recreate the links if necessary and then perform the following.
Delete the original file.
Check whether security_hard.txt can still be accessed.
Check whether security_soft.txt can still be accessed.
Use ls -l and readlink to investigate the symbolic link after deleting its target.
What is a dangling/broken symbolic link?
Does deleting the original filename immediately delete the underlying data when a hard link exists?
Justify your answer using the link count and inode.

```
hdoop@Ubuntu:~/Desktop/Assignment4/links$ rm incident.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$ cat security_hard.txt
Part D – Content Modification
Append the following line to security.txt:
Unauthorized login detected
Display the contents using all three filenames.
Is the newly added content visible through both links? Explain why.
Modify the file using security_hard.txt. Check the contents through security.txt and security_soft.txt. What do you observe?
Unauthorized login detected
Modifying security file
hdoop@Ubuntu:~/Desktop/Assignment4/links$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
```

Accessing via security_hard.txt: Still accessible.

- **Accessing via security_soft.txt:** Fails (broken link).
- **Dangling/Broken Link Definition:** A dangling symbolic link is a soft link that points to a target file or path that no longer exists in the filesystem.
- **Underlying Data:** Deleting the original filename does **not** immediately delete the underlying data when a hard link exists. The rm command removes the directory entry and decrements the inode's link count by 1. The operating system only frees the underlying data blocks when the link count reaches 0. Since **security_hard.txt** still exists, the link count is 1, and the data is preserved.

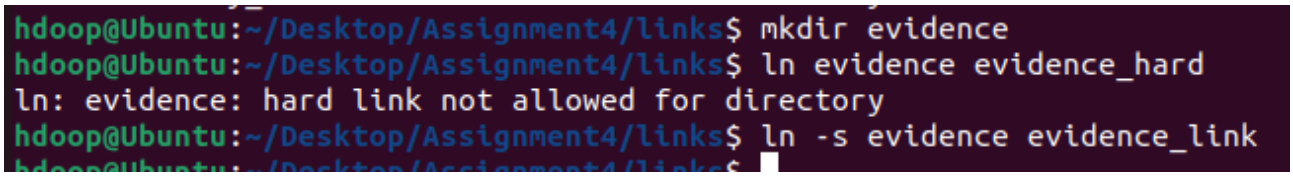
Part G — Directory and Filesystem Behaviour

Create a directory named evidence. Attempt to create a hard link to this directory. Record the result.
Create a symbolic link named evidence_link pointing to the evidence directory. Was the operation successful?
Based on your experiment, compare hard and symbolic links with respect to directory linking.
If multiple filesystems/mount points are available in the laboratory environment, attempt to create:
a hard link across filesystems
a symbolic link across filesystems

: Attempting to create a hard link to evidence fails with the error: hard link not allowed for directory.
Creating a symbolic link (evidence_link) to the directory succeeds.

Cross-Filesystem Linking :

- **Hard links:** Cannot be created across different filesystems or partitions because inode numbers are only unique within a single filesystem.
- **Symbolic links:** Can be successfully created across filesystems because they rely on absolute or relative path strings, not system-specific inodes.



Parameter	Hard Link	Symbolic Link
Inode number	Same as target	Unique (different from target)
File type	Regular file (-)	Symbolic link (l)
Link count	Shares target's link count (increments it)	Has its own link count (usually 1)
File size	Same as target	Size of the target's pathname string
Stores target pathname?	No	Yes
readlink output	Empty	Displays target path
Effect of modifying target	Updates reflect immediately	Updates reflect immediately
Effect of renaming target	Unaffected (still accessible)	Breaks (becomes a dangling link)

Parameter	Hard Link	Symbolic Link
Effect of deleting target	Unaffected (data remains accessible)	Breaks (becomes a dangling link)
Can become a dangling link?	No	Yes
Directory linking	Not allowed (for standard users)	Allowed
Cross-filesystem linking	Not allowed	Allowed

sort
Suppose alerts.txt contains:

192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8

Format:

IP-address username failed attempts

Sort according to failed attempts
Sort a list of suspicious IP addresses alphabetically.
Display only unique suspicious IP addresses using sort.
Sort a list of detected port numbers numerically.
Display port numbers in descending numerical order.
Sort usernames without considering uppercase/lowercase differences.
From a failed-login log, count how many times each IP address occurs and display the most frequent IP first.
Given IP,username,failed_attempts, sort records based on the number of failed attempts.
Sort a CSV vulnerability report based on severity score.
Extract IP addresses using grep, then use sort and uniq to identify unique IP addresses.
Create a pipeline that displays the top 5 IP addresses responsible for the most failed login attempts.

```
hdoop@Ubuntu:~/Desktop/Assignment4/links$ sort -k 3 -n alerts.txt
IP-address    username   failed attempts
172.16.0.5    guest      2
192.168.1.10  admin      5
10.0.0.20     admin      8
10.0.0.8      root       12
hdoop@Ubuntu:~/Desktop/Assignment4/links$ awk '{print $1}' alerts.txt | sort
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
IP-address
hdoop@Ubuntu:~/Desktop/Assignment4/links$ awk '{print $1}' alerts.txt | sort -u
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
IP-address
hdoop@Ubuntu:~/Desktop/Assignment4/links$ awk '{print $2}' alerts.txt | sort -f
admin
admin
guest
root
username
hdoop@Ubuntu:~/Desktop/Assignment4/links$ awk '{print $1}' alerts.txt | sort | uniq -c | sort -nr
      1 IP-address
      1 192.168.1.10
      1 172.16.0.5
      1 10.0.0.8
      1 10.0.0.20
hdoop@Ubuntu:~/Desktop/Assignment4/links$ sort -t ',' -k 3 -n alerts.csv
sort: cannot read: alerts.csv: No such file or directory
hdoop@Ubuntu:~/Desktop/Assignment4/links$ awk '{print $1}' alerts.txt | sort | uniq -c | sort -nr | head -n 5
      1 IP-address
      1 192.168.1.10
      1 172.16.0.5
      1 10.0.0.8
      1 10.0.0.20
hdoop@Ubuntu:~/Desktop/Assignment4/links$ grep -oE "\b([0-9]{1,3}\.){3}[0-9]{1,3}\b" logfile.txt | sort -u
grep: logfile.txt: No such file or directory
hdoop@Ubuntu:~/Desktop/Assignment4/links$ gedit logfile.txt
hdoop@Ubuntu:~/Desktop/Assignment4/links$ grep -oE "\b([0-9]{1,3}\.){3}[0-9]{1,3}\b" logfile.txt | sort -u
0.0.0.0
10.0.0.15
192.168.1.200
hdoop@Ubuntu:~/Desktop/Assignment4/links$
```